

Secure Trust OTA: A Resilient and Trust-Based OTA Framework for IoT Devices

25-26J-058

“Project Proposal Report”

Jayaweera N.S

Information Technology Specializing in Cyber Security
Department of Computer Systems Engineering,
Faculty of Computing,
Sri Lanka Institute of Information Technology

August 2025

Secure Trust OTA: A Resilient and Trust-Based OTA Framework for IoT Devices

25-26J-058

“Project Proposal Report”

Jayaweera N.S

Information Technology Specializing in Cyber Security
Department of Computer Systems Engineering,
Faculty of Computing,
Sri Lanka Institute of Information Technology

August 2025

Declaration

I declare that this is our work, and this proposal does not incorporate without acknowledgment of any material previously submitted for a degree or diploma in any other university or Institute of higher learning, and to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where the acknowledgment is made in the text. Also, I hereby grant to the Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic, or other mediums. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Jayaweera N S	IT22362780	Nuhansa J

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.


Mr. Kavinga Abeywardena:
Supervisor

10.09.2025
Date

.....
Ms. Dinithi Pandithage
Co-Supervisor

.....
Date

Abstract

In this era of rapidly expanding IoT deployments, IoT devices are highly vulnerable to unauthorized firmware updates and misuse of privilege because of their limited capabilities and remote operation. The secure OTA firmware update framework incorporates the Zero-Trust Authorization and policy enforcement Layer in the SecureOTA system to address this issue. Zero Trust layer makes sure that each update request is continuously authorized, verified, and authenticated without depending on implicit trust. This component includes features such as ; firmware attestation to verify update integrity of the firmware update prior to installation, Role- and policy-based access control to enforce update privileges and maintain access controls, and mutual TLS (mTLS) to authenticate both the device and server in order to mitigate security issues. The accountability and traceability of the OTA process is improved by logging and auditing the sessions, which helps to spot the root cause, just in case of an issue. Therefore, by implementing the Zero-Trust principles into the firmware update process, the system can reduce the possibility of insider threats, tampering, and unauthorized access while staying lightweight and better suited for IoT devices with a less amount of resources. Therefore, this method is a useful method to improve the security and Auditability of IOT appliances and devices, in which the reliable data and operational stability can be generates based on accurate device firmware.

Table of Contents

Abstract.....	3
List of Figures	6
List of Tables.....	6
Introduction.....	7
Background and Literature survey	8
Background	8
Literature review	9
Research Gap	11
Research problem.....	12
Objectives	13
Main objectives	13
Specific objectives.....	13
Methodology	14
Overall system.....	14
Individual system	16
Timeline.....	19
Project Requirements.....	19
Functional Requirements	19
Nonfunctional Requirements	21
User Requirements	22
System Requirements	22
Test Cases.....	23
Budget and budget justifications	24
Budget Justification	25
Commercialization: Use Cases.....	25
References.....	27

List of Figures

Figure 1: IOT Device update mechanism	8
Figure 2: System Architecture.....	14
Figure 3: Zero Trust Authentication Model	16
Figure 4: Timeline - Zero trust Layer.....	19

List of Tables

Table 1: Color indicators.....	14
Table 2: Testing and Validation.....	17

Introduction

In today's rapidly evolving technological environments starting from smart factories and remote sensors to healthcare monitoring and energy grids the Internet of Things (IoT) devices have become vital and plays an important part in some of the daily activities. These embedded systems need frequent Over-the-Air (OTA) firmware updates to introduce new features, patch vulnerabilities, and maintain the integrity in operations, especially when it costs a huge amount of money [1]. However, as over the air update mechanisms grows across different types of resource constrained devices, it introduces a considerable amount of critical security risks. A compromised update channel can lead to installing unauthorized firmware, device bricking, or breaches in system confidentiality [2]

As a solution, this project introduces SecureTrustOTA, a multi-layered framework which is designed to ensure that only authenticated, verified, and policy-compliant firmware updates are delivered and applied safely to the relevant IOT devices. The framework is built on four tightly integrated components. First, a Zero-Trust Authorization & Policy Enforcement Layer, that enforces continuous authorization through mutual TLS, dynamic policy-based access control, firmware attestation, and audit logging, ensuring only verified devices and users can access or execute updates. Secondly, a lightweight and secure over the air delivery mechanism which ensures efficient, encrypted, and resource-optimized update transfer using ECC and AES-CCM encryption, delta updates, and transmission strategies tailored for low-power devices. Third, a resilient, fail-safe update and rollback protection engine that safeguards device availability through dual-slot updates, rollback prevention, secure logging, and recovery scheduling to minimize downtime. And lastly, a threshold-based anomaly detection engine which continuously monitors key over the air parameters such as update size, memory usage, and timing to flag abnormal or malicious update behavior before it causes harm.

The proposal outlined in this document focuses on Zero-Trust Authorization & Policy Enforcement Layer, a central pillar of this framework, which is implemented to continually validate the devices and update integrity where no implicit trust is allowed.

This Zero-Trust component is a combined model of mutual TLS (mTLS) authorization [3], dynamic policy-based access control [4], firmware attestation [5], and audit logging to keep track of every firmware update interaction. According to modern cybersecurity principles, this architecture ensures that each device identity is authenticated in both directions, that update requests are explicitly authorized based on flexible policies, and that firmware integrity is always verified before the installation. At the end of every transaction (event) that occurs everything will be logged to maintain the accountability and for future forensic analysis.

By embedding Zero-Trust principles into OTA firmware delivery, it will help to lift the security but also demonstrates a practical application of secure communications, identity and access management, and attestation the core topics in Cybersecurity.

Background and Literature Survey

Background

The rapid expansion of the Internet of Things (IoT) has transformed sectors ranging from manufacturing and energy to healthcare and transportation, made smarter operations, real-time decision-making, and increased connectivity across devices and systems possible. These devices often require frequent firmware updates to patch vulnerabilities, improve performance, and introduce new features. The scalability, affordability, and remote update delivery features of over-the-air (OTA) update mechanisms have made them the most popular approach for firmware management and distribution.

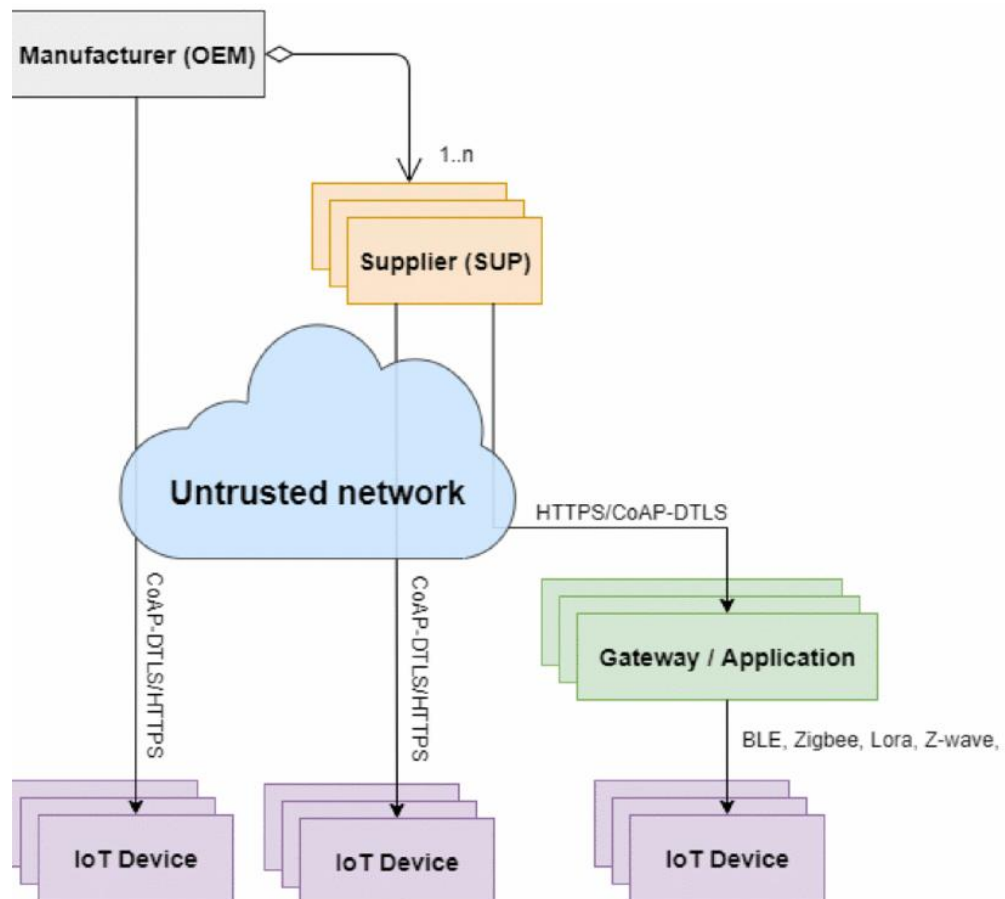


Figure 1: IOT Device update mechanism

However, over the air updates also expose IoT infrastructures to significant security risks. Malicious actors can exploit unprotected update channels to inject tampered firmware, impersonate legitimate devices, or intercept and alter communications during transmission. Such compromises can lead to operational disruptions, data exfiltration, and even the weaponization of IoT devices in coordinated cyberattacks. [6]

Traditional security models, which rely heavily on perimeter defenses, are less adequate and useful in these hugely distributed environments. This has led to the adoption of Zero-Trust security principles, which operate under the rule of “never trust, always verify.” In the context of over the air firmware

delivery, Zero-Trust ensures that every interaction, from initiating an update request to installing firmware is authenticated, authorized, and verified.

Within the OTA update process, Zero-Trust principles can be operationalized through a combination of security mechanisms. Mutual Transport Layer Security (mTLS) which ensures bidirectional authentication between the device and the update server, prevents impersonation and man-in-the-middle attacks, and ensures secure, encrypted communications between the two ends. Role and policy based access control enables fine grained authorization, ensuring that only explicitly permitted update actions are executed on specific device groups. Firmware attestation further improves the security by verifying the integrity and authenticity of firmware before installation [7], while audit logging provides traceability for all update-related events, supporting both compliance and post-incident forensics.

By integrating these mechanisms into the over the air delivery workflow, a Zero-Trust authorization and policy enforcement system can significantly strengthen the security posture of IoT device management. This approach moves beyond reactive defenses, embedding trust validation at every stage of firmware delivery to ensure that updates are both authorized and verifiably safe.

Literature Review

Saad El Jaouhari (2022) targets insecure OTA updates on highly constrained IoT devices by proposing a SecFOTA architecture. The paper defines roles (Manufacturer, Supplier, Gateway, IoT Device) and a trust chain for firmware delivery [8]. Its approach uses public-key signatures and secure channels: the gateway verifies signed manifests and firmware images, and can perform mutual authentication (e.g. mTLS) with suppliers to ensure end-to-end integrity [8]. It also includes an attestation step, where the IoT device sends a “proof” of successful update back to the gateway [8]. Strengths of this design include use of a gateway to offload security checks from tiny devices and the layered chain of trust from manufacturer to device. However, limitations include the fact that many aspects (e.g. key-establishment protocols) are left to future work, and no performance evaluation is provided [8]. In practice, the Zero-Trust OTA layer shares the goal of ensuring only authorized updates reach devices but goes beyond by enforcing dynamic access policies and comprehensive audit logging, which are absent in SecFOTA (Secure firmware over the air). This continuous verification adds resilience against insider misuse and evolving threats that static trust chains alone cannot handle.

N. Asokan et al. (2018) present ASSURED, a secure firmware-update framework for large-scale IoT deployments. They observe that typical update systems (e.g. TUF, Uptane) assume direct device interaction and no broadcast, so ASSURED instead identifies all stakeholders (Manufacturer, Controller, Device) and introduces an authorization token bound to each firmware package [9]. In their method, the manufacturer signs an update and includes a token encoding device-specific constraints; a local Controller (e.g. gateway) then relays the update to devices. Each device can verify the manufacturer’s token without needing online contact with the Manufacturer [9]. A notable feature is that after installation the device must return an attestation of its software state to the Controller, so the controller can confirm the update succeeded [9]. ASSURED’s strengths are end-to-end authenticity, explicit update constraints, and minimal trust assumptions: it offloads heavy crypto to a controller or trusted hardware (HYDRA OS or ARM TrustZone) to suit constrained devices. Its weaknesses include relying on specialized security

architectures and static tokens (lacking per-update policy flexibility), and it does not inherently provide audit logs or continuous attestation during operation. Our Zero-Trust OTA layer similarly provides manufacturer-issued authorization for updates and requires device state attestation, but extends ASSURED by introducing real-time, policy-based authorization and comprehensive logging for each OTA transaction. This directly addresses the rigidity and limited adaptability of ASSURED's token-based approach and increases the security and auditability.

Meha James et al. (2025) proposes a Zero Trust architecture for general IoT networking (specifically, MQTT messaging). They embed an ABAC (Attribute-Based Access Control) engine inside the MQTT (Message Queuing Telemetry Transport) broker as a Policy Enforcement Point (PEP) [10]. The broker evaluates each publish/subscribe request against fine-grained policies (using Axiomatics ALFA) and maintains mutual TLS connections, achieving continuous verification of client identities. In experiments on an IoT cyber-range, this in-broker enforcement adds only ~20 - 45 ms of latency (vs. >100 ms for traditional setups) [10]. The approach's novelty is applying Zero-Trust ABAC directly to the MQTT layer, greatly improving scalability and ensuring no implicit trust in connected devices. However, it targets the communication layer (MQTT topics) rather than firmware updates, so it does not address firmware authenticity or attestation. Its relevance is that it shows how embedding PEP (Policy enforcement point) /PDP (Policy decision point) logic enforces least-privilege access with low latency. In SecureOTA context, this system similarly enforces continuous, attribute-based authorization (e.g. only devices with proper credentials can download firmware), though specifically at the firmware-delivery level. This highlights our novelty: extending Zero-Trust policy enforcement beyond message exchanges into update distribution, where the risks of compromise are significantly higher.

Stefan Hristozov et al. (2018) tackle on-device runtime attestation for tiny IoT microcontrollers. They present a DICE-based scheme where the device isolates an attestation routine in privileged mode using a standard MPU (Memory Protection Unit) and lockable ROM secret [5]. When the verifier requests attestation, the MCU executes the isolated code to measure its firmware (hash) and returns a signed report. Crucially, this method requires no special hardware extensions beyond a secure boot secret and basic CPU features; it works on commodity MCUs (demonstrated on an STM32L476) [5]. Its strength is feasibility on low-cost devices without TPM or TrustZone. Limitations include the need for a DICE-style boot-loader and that it only generates attestations, without addressing how updates are fetched, authorized, or logged. In relevance to our system, this scheme could be used as the device's mechanism to prove it is running unmodified firmware. Our Zero-Trust OTA layer incorporates similar runtime attestation but strengthens it by embedding it in a broader update framework: attestation is combined with mutual TLS, dynamic policy enforcement, and audit logging, thereby mitigating the isolation of Hristozov's approach.

Chae-Yeon Park et al. (2025) propose a lightweight FOTA scheme to thwart man-in-the-middle attacks on firmware downloads. Their method applies server-side DEFLATE compression plus a dual-key XOR cipher on the firmware, and splits the payload across multiple transmission channels [11]. This reduces the firmware size and encryption overhead while ensuring that even if one channel is compromised, the full firmware cannot be reconstructed by an attacker [11]. In testing, the authors report significantly lower latency, memory usage, and power consumption compared to uncompressed or single-channel approaches, yet still resist brute-force interception [11]. The novelty lies in this combined use of

compression and multi-channel XOR encryption for resource-constrained IoT. However, the scheme is narrow in scope: it focuses on confidentiality and performance but does not handle device authentication, authorization, or trust management. For our Zero-Trust OTA layer, such a technique could enhance data security (making update payloads smaller and harder to tamper), but our system add the missing critical layers such as mutual authentication, access policy enforcement, and continuous auditability for ensuring that confidentiality is complemented by verifiable trust and control.

Chunwen Liu et al. (2024) provide a broad survey of Zero Trust in IoT contexts. They analyze the research landscape and discuss how ZT principles (never trust, always verify) apply to IoT's heterogeneous devices [12]. The paper identifies key ZT components, e.g. continuous device authentication, Policy Enforcement Points (PEP) and Decision Points (PDP), and asset-centric verification, in an IoT setting [12]. It notes that IoT deployments need agent-centric trust evaluation, fine-grained access policies, and regular re-evaluation of device trust levels. As a review, its strengths lie in summarizing many approaches and highlighting open challenges. It does not present new firmware-specific mechanisms, however. Its findings underscore the need for continuous authentication and policy control, exactly the aim of our OTA authorization layer. In other words, Liu et al. show why every IoT update interaction should be authenticated, authorized (via PEP/PDP), and audited. Our system implements this by requiring mTLS client certificates, evaluating a dynamic access policy for firmware delivery, and logging each update event in line with these zero-trust guidelines [12]. Unlike the survey, which stops at conceptual principles, our work operationalizes them in a concrete OTA framework, bridging the gap between abstract zero-trust requirements and practical, device-level update enforcement.

Research Gap

All the existing secure OTA update mechanisms for IoT devices, such as Uptane and TUF, are focused on ensuring firmware integrity and authenticity but not real-time, per-request device verification and adaptive access control. Current authorization approaches are static in nature, which allows updates to be delivered to any enrolled device regardless of its compliance state or security posture. While device attestation techniques exist, they are rarely integrated directly into the OTA authorization process, and mutual TLS (mTLS) enforcement is not consistently applied. Furthermore, audit logging of update transactions is often minimal, reducing Auditability and forensic capability.

Zero trust authorization and policy enforcement layer address these limitations by embedding Zero-Trust principles into OTA firmware delivery. The proposed component combines mandatory mTLS authentication, dynamic role- and policy-based access control, firmware attestation, update verification, and tamper-evident audit logging. Unlike prior solutions, the novelty of this approach lies in its lightweight adaptation of mTLS and signature/hash verification for constrained IoT devices, reducing both handshake latency and computational overhead while maintaining strong authentication guarantees. Additionally, dynamic, per-request policy enforcement and integrated firmware attestation ensure that each update transaction is continuously validated against the device's real-time trust status, rather than relying on static enrollment. Finally, the inclusion of tamper-evident yet lightweight audit logging enhances forensic capability without imposing heavy storage or processing costs. Together, these innovations make the Zero-Trust Authorization Layer not only more secure but also more practical and efficient for resource-limited IoT environments, representing a clear advancement over existing OTA security frameworks.

The central research gap addressed in this work is the lack of effective mechanisms that improves both security and auditability in resource constrained IoT environments. Existing OTA frameworks focus mainly on authentication and integrity but neglect efficient, tamper-evident auditability, which limits their ability to support incident response and forensic analysis. By incorporating lightweight yet secure audit logging, the proposed system not only strengthens overall security but also provides verifiable traceability of update processes without imposing prohibitive overheads. Consequently, this research advances over the air security frameworks by delivering a more secure, auditable, and practical solution for constrained IoT devices.

In summary, through the integration of lightweight zero-trust principles, optimized cryptographic protocols, and tamper-evident audit mechanisms, this research aims to achieve significantly enhanced security and auditability compared to previous OTA systems, while maintaining feasibility for resource-constrained IoT environments.

Research problem

Despite advancements in secure OTA firmware delivery for IoT devices, current solutions remain vulnerable to unauthorized updates, compromised devices, and insufficient verification of trust status during update transactions. Traditional methods rely heavily on static trust models, where devices, once enrolled, are assumed to remain trustworthy indefinitely. This creates security blind spots, as compromised or non-compliant devices can still receive critical updates, which might potentially lead to further exploitation.

Moreover, while existing protocols like Uptane and TUF provide robust integrity and authenticity checks, they often lack integration with real-time device authentication, mutual TLS enforcement, dynamic role- and policy-based access control, and continuous firmware attestation. Audit logging is also a loophole at present, and it's frequently incomplete or non-tamper-resistant, which limits post-incident forensics. The problem is the absence of a comprehensive, integrated Zero-Trust authorization and policy enforcement framework for OTA updates that enforces per-transaction verification, adaptive access policies, and tamper-evident audit trails, capabilities essential to counter advanced threats targeting IoT firmware supply chains.

Objectives

Main objectives

The objective of this project is to create a secure and resilient OTA update framework for IoT devices by integrating Zero-Trust authorization, lightweight secure delivery, fail-safe rollback protection, and anomaly detection. The system ensures only verified updates reach devices, firmware is delivered efficiently, failures or attacks are safely recovered from, and abnormal behaviors are detected early making IoT updates trustworthy, resilient, and future proof.

Specific objectives

- **Mutual TLS (mTLS) for OTA Firmware Delivery:** Implement mutual TLS between IoT devices and the OTA update server to ensure secure, encrypted, and authenticated communication. It will be used to validate that only authenticated endpoints can initiate or receive update transactions.
- **Role- and Policy-Based Access Control:** Design and enforce dynamic, context-aware access policies for update distribution. It restricts firmware delivery based on device roles, compliance status, and operational context. In addition, the policies ensure that only authorized administrators are permitted to initiate and send firmware updates, which prevents unauthorized users from distributing potentially malicious or unverified updates.
- **Firmware Attestation and Verification:** Integrate firmware attestation mechanisms to validate device integrity before delivering updates. And it also performs update verification to ensure firmware authenticity and integrity before installation.
- **Audit Logging and Forensics:** Maintain tamper-evident audit logs of all update transactions for forensic analysis and compliance reporting. It enables post-incident investigations by storing detailed records of update requests, authentication events, and policy decisions.

Methodology

Overall system

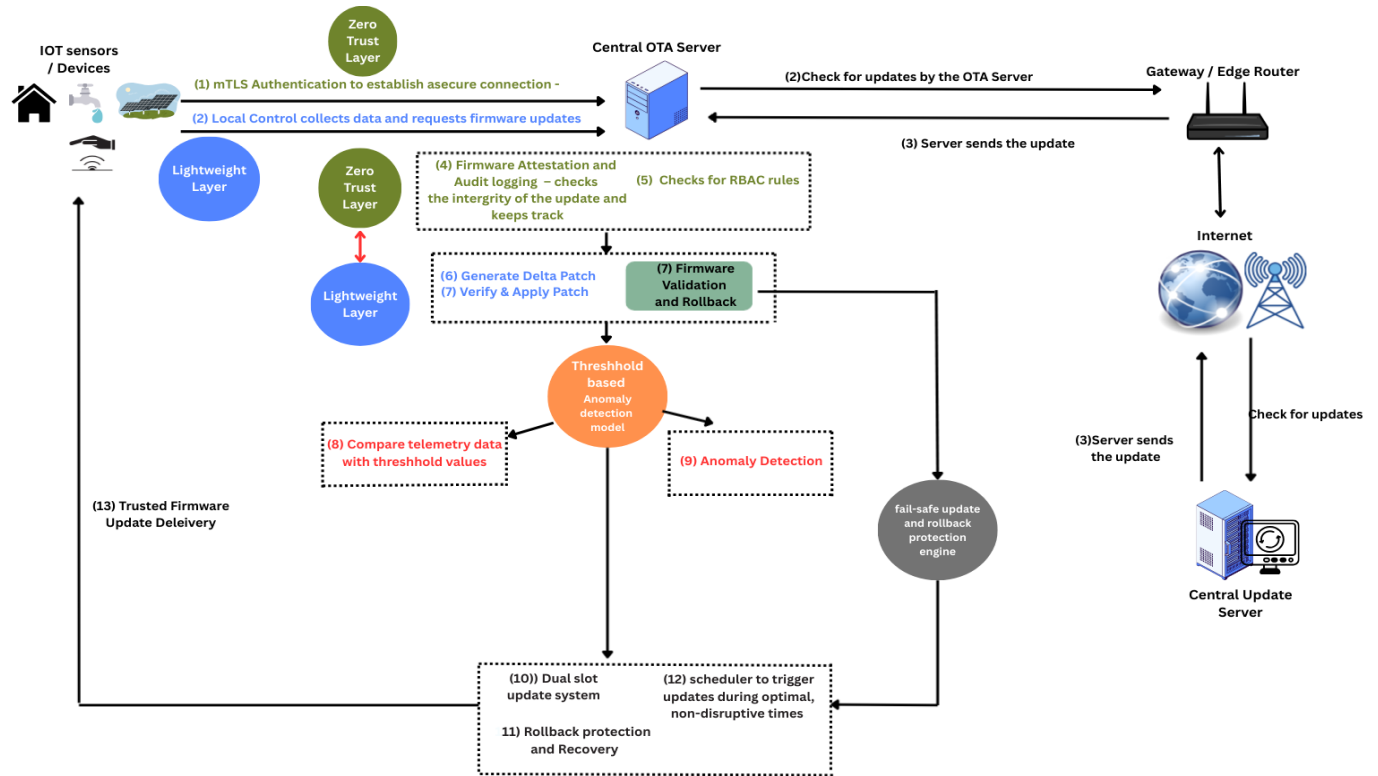


Figure 2: System Architecture

The overall system works with the corporation of 4 layers indicated in 4 colors.

Table 1: Color indicators

Color	Layer Number	Indicated Layer
Green	Layer # 1	Zero-Trust Authorization and policy enforcement Layer
Blue	Layer #2	Lightweight Secure OTA Core
Orange	Layer #3	Threshold-Based Anomaly Detection Engine
Grey	Layer #4	Resilient Fail-Safe Update Engine

- Layer #1: Zero-Trust Authorization Layer

The system begins by enabling strict identity verification through mutual TLS (mTLS), where both the device and the update server authenticate each other to ensure only trusted parties can communicate. Once the connection is established, role based and policy-based permissions control what actions each device is authorized to perform and by whom to receive the updates, such as requesting or applying updates. Before any installation takes place, firmware attestation mechanisms verify the legitimacy and integrity of the update package and the device, ensuring it has not been tampered with or replaced by malicious code. Finally, all activities are continuously logged and audited, providing a complete trace of the update process for accountability, compliance, and forensic analysis.

The flow: Authenticate → Authorize → Verify → Log & Audit

- Layer #2: Lightweight Secure OTA Core

The process begins with the Local Control unit, which is responsible for gathering device data and initiating firmware update requests. Once an update is available, the Delta Patch Generator computes the difference between the existing firmware and the new version, producing a lightweight patch. This patch is then encrypted using ECC and AES-CCM to ensure both confidentiality and integrity during transmission. Before installation, the Verify & Apply Patch stage validates the authenticity of the update, making sure it has not been altered or injected with malicious code. Following this, Firmware Validation and Rollback mechanisms safeguard the device by running health checks on the new firmware; if any failures or instabilities are detected, the system automatically reverts to the last stable version. Finally, the Alert Dashboard provides visibility to administrators by connecting back to the central server and displaying anomalies or unusual activities detected throughout the update process.

The flow: Collect → Generate & Encrypt → Verify & Apply → Validate / Rollback → Alert & Monitor.

- Layer #3: Threshold-Based Anomaly Detection Engine

IoT devices continuously send telemetry data to the central OTA server through log forwarding agents, allowing the system to track their operational status. The received data is compared against predefined threshold values stored in the database to determine whether the device is functioning within acceptable limits. If the readings fall within these thresholds, the update process proceeds as expected. However, if any anomalies are detected such as abnormal battery levels, excessive CPU usage, or updates requested outside the permitted time window an alert is automatically generated and displayed on the monitoring dashboard. This ensures that only healthy and stable devices receive updates, while potential risks are flagged for further investigation.

The flow: Telemetry → Compare with thresholds → If normal, update → If abnormal, alert & monitor.

- Layer #4: Resilient Fail-Safe Update Engine

To ensure resilience during firmware updates, the system is designed to guarantee that IoT devices can always recover safely. When a new firmware is downloaded, it first undergoes an integrity check to confirm that it is valid and untampered. Once verified, the firmware is stored in a secondary partition,

known as the staging area, without disturbing the device's current working version. The bootloader then switches to the new firmware while running continuous health checks to monitor system stability. If any issues arise such as failures, instability, or corruption of the system automatically rolls back to the previous working firmware, ensuring uninterrupted functionality. This process prevents device bricking and safeguards reliability, even in cases of power loss or faulty updates.

The flow: Update → Verify → Test → Rollback if fail → Keep running if pass.

Individual system

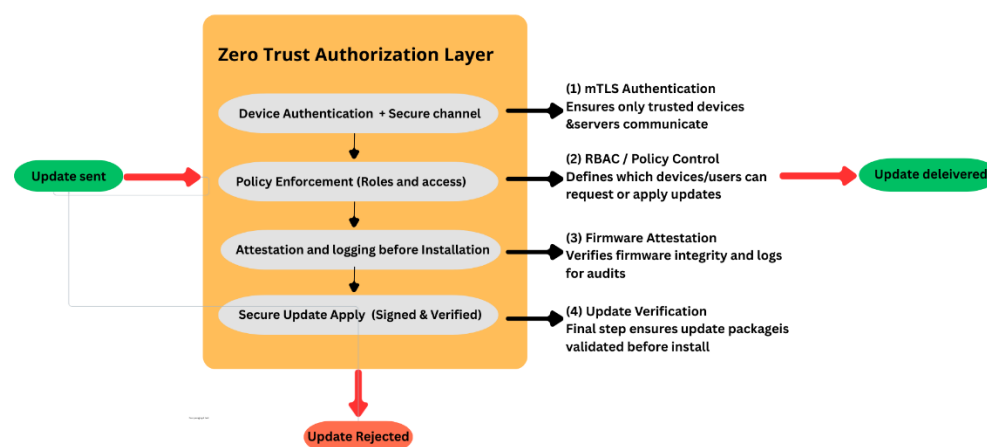


Figure 3: Zero Trust Authentication Model

The Zero-Trust Authorization Layer enhances the security of OTA (Over-the-Air) firmware updates by ensuring that no device, user, or service is trusted implicitly. Instead, it uses strong authentication and continuous verification at every stage. The core mechanisms inside this component include mutual TLS (mTLS) authentication, Role- and policy-based access control, firmware attestation and e device before updates are applied, while audit logging maintains a traceable record of all update-related actions for accountability and forensic analysis. By embedding Zero-Trust principles directly into the firmware update delivery process, this component ensures robust protection against supply-chain attacks and unauthorized modifications

When an update is sent, it first passes through the Zero Trust Authorization Layer. The process begins with Device Authentication using mutual TLS (mTLS), where both the device and server prove their identities and establish an encrypted channel. This ensures that only trusted endpoints can communicate securely. Next, Policy Enforcement checks whether the device is authorized to receive the update. Access decisions are made dynamically based on roles and attributes such as compliance status, location, or operational context, preventing unauthorized devices from applying updates.

If the request is allowed, the system performs Firmware Attestation and Logging. The device proves its current firmware integrity while all actions including authentication, policy checks, and attestation results are recorded in tamper-proof audit logs for traceability and compliance. Finally, the Secure Update Apply

step validates that the firmware package is digitally signed and verified against trusted keys. Only if this check passes is the update installed; otherwise, it is rejected.

The Zero-Trust Authorization Layer strengthens the OTA (Over-the-Air) firmware update process by ensuring that no entity, device, user, or services trusted (by default) , as mentioned previously. Therefore, every access and update action has to go through a strict layer of authentication, authorization, and integrity validation. The component integrates several mechanisms, each with clear validation procedures to guarantee the strength. The below table shows how each sub component is tested and validated in order to make the update process more secure.

Table 2: Testing and Validation

Security Feature / Subcomponent	Process	Testing and Validation mechanism
Device Authentication with Mutual TLS (mTLS)	When a device attempts to connect to the update server, an mTLS handshake is initiated. Both server and device present X.509 certificates and validate each other's trust chains. Only after a valid exchange is an encrypted channel established.	• Attempt connections with expired, revoked, or self-signed certificates to confirm rejection. • Simulate simultaneous device authentications under load to ensure performance is not degraded. • Verify cipher suite negotiation to ensure only strong cryptography (e.g., TLS 1.3 with AES-CCM/ECC) is allowed.
Role- and Policy-Based Access Control (RBAC/PBAC)	Once authenticated, a device's permissions are validated against dynamic access policies. Policies may be based on roles (e.g., "sensor," "gateway") or contextual attributes (e.g., device compliance status, geographic location).	• Attempt unauthorized update requests to confirm whether it denies. • Modify access rules dynamically and ensure enforcement occurs without requiring downtime. • Stress-test multiple concurrent policy evaluations to confirm correct enforcement under load.
Firmware Attestation	Before applying any update, the system verifies the integrity and authenticity of the firmware. Cryptographic hashes (e.g., SHA-256) of the package are computed and compared with trusted reference values signed by the manufacturer.	• Upload a tampered firmware (hash mismatch) and confirm rejection. • Test signature validation using trusted vs. untrusted public keys. • Validate attestation under poor network conditions to ensure system resilience.
Audit Logging	Every critical action an authentication attempts, policy enforcement results,	• Attempt unauthorized log access or modification and verify detection/prevention.

	attestation outcomes, update status is logged in a secure, tamper-proof system. Logs are time-stamped and cryptographically protected.	• Validate that logs capture failed and successful events consistently. • Run integrity checks (e.g., hash chaining or Merkle tree) to ensure logs cannot be altered without detection.
Secure Update Apply	After attestation, the firmware package is validated against trusted digital signatures. Only if the cryptographic signature verification passes is the update applied; otherwise, the update is rejected.	• Test update application with unsigned and incorrectly signed firmware to confirm rejection. • Validate rollback protection by attempting to install an older version (downgrade attack). • Measure update application time under different package sizes to ensure performance targets are met.

Timeline

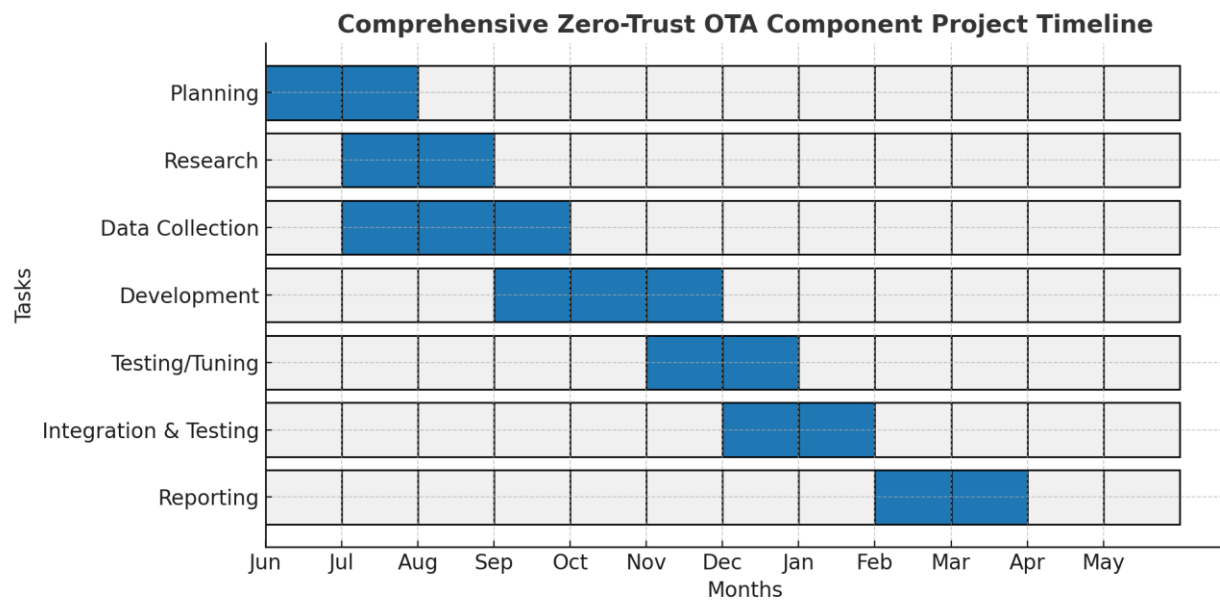


Figure 4: Timeline - Zero trust Layer

Project Requirements

Functional Requirements

1. Mutual TLS (mTLS) Communication
 - The system must establish secure communication channels between IoT devices and the update server with the use of mutual TLS. Here both the client and server must authenticate each other using valid X.509 certificates before any data exchange takes place. The system rejects any connection where the certificate is expired, revoked, or untrusted.
2. Role- and Policy-Based Access Control (RBAC / PBAC)
 - The system must support role-based and policy-based access control to determine the update permissions. Access rules must be dynamically updatable without requiring device downtime. The system must enforce the principle of least privilege, which grants devices and users only the permissions required for their specific roles.

3. Firmware Attestation

- The system must verify the integrity and authenticity of firmware before installation. Attestation must use cryptographic hashes and digital signatures. The firmware source must be validated against a trusted manufacturer or repository.

4. Device Authentication

- Each IoT device must be uniquely identifiable and authenticated before accessing update services. Authentication must be continuous (not just at the connection start), in line with Zero Trust principles. The system must store and manage device identities in a secure identity management system.

5. Update Verification

- The system must verify that downloaded firmware matches the expected checksum/hash before installation. Updates must be atomic it means, either fully completed and verified or rolled back without partial installation. All update packages must be timestamped and version-controlled to prevent downgrade attacks.

6. Audit Logging

- The system must maintain logs for all update-related events, including:
 - Device authentication attempts
 - Update requests
 - Verification results
 - Policy changes
- Logs must be securely stored and protected from tampering. The system must allow authorized auditors to review logs in real-time or historically.

7. Scalable Update Delivery

- The system must support secure, simultaneous updates to multiple devices while maintaining mTLS for each session. Update delivery must be optimized to handle network interruptions gracefully.

Nonfunctional Requirements

- Security
 - The system must enforce mTLS with 2048-bit RSA / ECC-256 certificates for all connections.
 - All firmware must be signed digitally (SHA-256 or higher) and verified before installation.
 - Unauthorized access attempts must be logged with 100% coverage of authentication failures.
- Performance
 - The system should support at least 100 device update requests without noticeable reduction in response time.
 - Firmware verification (checksum + signature validation) must complete within < 2 seconds for files ≤ 50 MB.
 - Policy enforcement decisions must execute in < 500 ms per request.
- Scalability
 - The system should be able to scale horizontally to support at least 1,000 devices without requiring architectural changes.
 - The system should handle a firmware update load of at least 50 devices per minute under normal network conditions.
- Availability & Reliability
 - The OTA update service should have an uptime of $\geq 99\%$ during operation.
 - In case of failure, rollback or recovery should occur automatically within ≤ 60 seconds.
 - Update packages must be delivered with $\geq 99.5\%$ reliability (minimal packet loss or corruption).
- Maintainability
 - Code modules should follow modular design, with changes in one component requiring $\leq 10\%$ code modification in dependent modules.
 - Configuration updates (e.g., policy changes) must be applied without system restart in $\geq 90\%$ of cases.
- Auditability
 - All critical events (auth, update requests, policy changes, rollbacks) must be logged with 100% coverage.
 - Logs must be queryable within ≤ 2 seconds for the most recent 1,000 events.
 - Logs must be retained securely for at least 180 days.

- Usability

- The web dashboard must load in ≤ 3 seconds under normal network conditions.
- Admins should be able to initiate a firmware update with ≤ 3 clicks.
- Notification of update success/failure must be delivered to users within ≤ 5 seconds of the event.

User Requirements

- Users should be able to access the OTA update system through a web-based dashboard.
- Users (Admin) should be able to upload new firmware updates securely.
- Users should be able to manage authorization policies (zero-trust rules).
- Users should be notified if an update fails or is blocked.

System Requirements

- Operating System: The system should support either Linux (e.g., Ubuntu, Debian) or Windows (Windows 10/11, Windows Server) in order to host and run the platform.
- Minimum 2 GB RAM and 10 GB storage per VM (scalable depending on IoT device load).
- Network connectivity with secure communication channels (TLS/DTLS).
- Programming Languages: Python (for anomaly detection engine + logic), Go/Java (for OTA delivery service), Bash (for automation scripts).
- Web Server: Apache/Nginx for hosting the central dashboard.
- Database: PostgreSQL or MongoDB for storing update logs, device identities, and policies.
- Security Libraries: OpenSSL or mTLS frameworks for Zero-Trust enforcement.
- Packet Capture/Monitoring Tools: Wireshark, tcpdump, or equivalent for monitoring OTA traffic.
- Container/Orchestration Tools (Optional): Docker/Kubernetes for scalable deployment.

Test Cases

1.Subcomponent #1: Mutual TLS (mTLS) Implementation

Test Case	Expected Result
Verify that devices with valid client certificates can establish a secure connection.	Connection established successfully
Verify that devices with invalid/expired certificates are rejected.	Connection denied with error log
Test certificate revocation by attempting to connect with a revoked certificate.	Connection rejected
Test fallback attempts (e.g., plain TLS without client auth).	System enforces mTLS only, fallback is blocked.

2.Subcomponent #2: Role-/Policy-Based Access Control (RBAC/PBAC)

Test Case	Expected Result
Verify that devices with assigned roles can only access permitted resources	Authorized actions succeed; unauthorized actions fail
Attempt privilege escalation (low-privilege device trying admin action).	Access denied and event logged
Modify policy dynamically (e.g., revoke access in real time).	Device loses access immediately.
Test multiple simultaneous policy checks under load	System enforces policies without performance degradation

3. Subcomponent #3 and #4: Firmware Attestation & Audit Logging

Test Case	Expected Result
Verify firmware integrity attestation during OTA update	Only unmodified firmware passes attestation
Test tampered firmware file delivery	Update rejected and event logged
Verify audit log generation for all access attempts.	Every authorization attempt has a timestamped log entry.
Test log tampering (try modifying/deleting entries	Logs remain immutable or system detects tampering

4. Authentication of Devices

Test Case	Expected Result
Verify that only registered devices can authenticate	Unregistered device authentication fails
Attempt repeated failed authentications (brute force).	Device is blocked after threshold attempts
Test device re-authentication after certificate renewal	Authentication succeeds with new valid cert.

Budget and budget justifications

Item	Quantity	Unit Cost (LKR)	Total Cost (LKR)	Justification
Laptop/PC (Already existing)	-	-	0	Already available personal resources.
IoT Development Boards (e.g., ESP32 / STM32)	4	6,500	26,000	ESP32 boards are cost-effective and widely used in IoT security research.
Secure Boot/Encryption Modules (e.g., ATECC608A Crypto chip)	2	3,500	7,000	Used to simulate hardware-based cryptographic security and support ECC features.
Storage Devices (USB drives / SD cards)	4	1,000	4,000	For transfer firmware, test data storage, and recovery testing.
Networking Devices (Wi-Fi Router / Switch)	1	8,500	8,500	Required to simulate enterprise-like IoT device connectivity for OTA delivery.
Software Tools & Libraries	-	-	0	Most of the development tools (Arduino IDE, PlatformIO, OpenSSL, Wireshark, etc.) are open-source.
Cloud Resources (Testing - Free Tier)	-	-	0	Cloud services such as AWS IoT Core / Azure IoT Hub will be used within free student credits or free tier.
Printing & Documentation	-	-	3,000	Printing of project proposal, final report, and Other essential materials.

Cables, Connectors, Power supplies	-	-	5,000	To power-up and connecting development boards, routers, and test setups.
--	---	---	-------	--

Total Estimated Cost: LKR 53,500

Budget Justification

The budget has been structured with careful consideration to remain both cost-effective and practical for the Sri Lankan academic setting.

- **Development Boards:** Low-cost ESP32/STM32 boards are selected as they provide sufficient processing power to emulate IoT devices and enable OTA update testing. To ensure smooth development, each student will be provided with an individual board for independent use.
- **Cryptographic Modules:** A small number of ATECC608A secure elements are included to demonstrate hardware-based cryptography in practice. This keeps costs manageable while still reflecting real-world security features.
- **Network Setup:** A shared Wi-Fi router will function as the central testbed, creating a realistic environment for carrying out OTA experiments under typical IoT networking conditions.
- **Software Resources:** Open-source development platforms and cryptographic libraries will be utilized, keeping software-related expenses at zero.
- **Cloud Infrastructure:** Free-tier services and student plans from cloud providers will support the simulation of an OTA delivery environment, removing the need for paid subscriptions.
- **Miscellaneous & Printing:** A small allocation is reserved for essential printing, documentation, and other minor project-related requirements.

Commercialization: Use Cases

The SecureTrustOTA framework has a wide-ranging application across multiple sectors where secure and reliable device updates are critical. By ensuring minimal downtime and trusted firmware delivery, it reduces operational costs and mitigates risks associated with device failures. Key use cases include:

- **Smart Metering & Energy Grids:** Utilities can deploy millions of smart meters with confidence, knowing that updates will not disrupt billing or regulatory compliance. Our framework ensures continuous operation and minimizes the need for costly technician interventions, helping maintain smooth energy distribution and accurate data collection.
- **Healthcare IoT Devices:** Critical medical monitoring equipment, such as wearable devices and bedside IoT sensors, requires uninterrupted operation. By providing resilient, trust-verified updates, our framework reduces the risk of service failures that could compromise patient safety, ensuring consistent and reliable healthcare monitoring.

- **Industrial IoT (IIoT):** SCADA systems, manufacturing plants, and other industrial operations demand 24/7 uptime. The lightweight and resilient nature of our OTA framework allows secure updates without interrupting essential production processes, enhancing operational efficiency and reliability.
- **Smart Homes & Consumer Devices:** Everyday IoT devices, such as smart plugs, cameras, and appliances, benefit from seamless updates that guarantee device safety. Customers can confidently update their devices without the risk of malfunctions, improving user trust and satisfaction.

References

- [1] K. Arakadakis *et al.*, *Firmware Over-the-Air Programming Techniques for IoT Networks—A Survey*. *ACM Computing Surveys*, vol. 54, no. 9, pp. 1–36, New York, NY, USA: ACM, 2021.
- [2] J. L. Hernández-Ramos *et al.*, *Updating IoT Devices: Challenges and Potential Approaches*. In *Proc. Global Internet of Things Summit (GloTS)*, Dublin, Ireland: IEEE, 2020, pp. 1–6.
- [3] J. Hallin, *Evaluation of TLS and mTLS in Internet of Things Systems*. M.S. thesis, 2025.
- [4] M. Karimibiuki *et al.*, *Dynpolac: Dynamic Policy-Based Access Control for IoT Systems*. In *Proc. 23rd IEEE Pacific Rim Int. Symp. Dependable Computing (PRDC)*, Taipei, Taiwan: IEEE, 2018, pp. 94–103.
- [5] S. Hristozov *et al.*, *Practical Runtime Attestation for Tiny IoT Devices*. In *Proc. NDSS Workshop on Decentralized IoT Security and Standards (DISS)*, San Diego, CA, USA: Internet Society, 2018.
- [6] S. El Jaouhari and E. Bouvet, *Secure Firmware Over-the-Air Updates for IoT: Survey, Challenges, and Discussions*. *Internet of Things*, vol. 18, p. 100508, Amsterdam, Netherlands: Elsevier, 2022.
- [7] A. Kolehmainen, *Secure Firmware Updates for IoT: A Survey*. In *Proc. IEEE Int. Conf. Internet of Things (iThings), GreenCom, CPSCom, SmartData*, Halifax, NS, Canada: IEEE, 2018, pp. 1120–1126.
- [8] S. El Jaouhari, *Toward a Secure Firmware OTA Updates for Constrained IoT Devices*. In *Proc. IEEE Int. Smart Cities Conf. (ISC2)*, Paphos, Cyprus: IEEE, 2022, pp. 1–7.
- [9] N. Asokan *et al.*, *ASSURED: Architecture for Secure Software Update of Realistic Embedded Devices*. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2290–2300, Piscataway, NJ, USA: IEEE, 2018.
- [10] M. James *et al.*, *Authentication and Authorization in Zero Trust IoT: A Survey*. In *Proc. 35th Irish Signals and Systems Conf. (ISSC)*, Belfast, UK: IEEE, 2024, pp. 1–6.
- [11] C.-Y. Park, S.-J. Lee, and I.-G. Lee, *Secure and Lightweight Firmware Over-the-Air Update Mechanism for Internet of Things*. *Electronics*, vol. 14, no. 8, p. 1583, Basel, Switzerland: MDPI, 2025.
- [12] C. Liu *et al.*, *Dissecting Zero Trust: Research Landscape and Its Implementation in IoT*. *Cybersecurity*, vol. 7, no. 1, p. 20, London, UK: Springer, 2024.